

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1-CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	100
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	M. Nivedha	Reg.No.:	192471040
Department:	CS&BS	Date:	

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

83%
48

Code of Conduct:

I M. Nivedha (192471040) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1	2	2	2	1.5	1
Q2	2	2	2	1.5	1
Q3	2.5	2.5	1.5	1	0.5
Q4	2	2.5	2	1	0.5
Q5	2.5	2	2.5	1.5	1

RUBRICS FOR CALCULATION:

41.5

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25% 2.5	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25% 2.5	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25% 2.5	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15% 1.5	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10% 1	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	C05	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	100
C05 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A
Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

I/O Communication Mechanism for a Real-Time Embedded System

1. Problem Understanding

The real-time embedded system receives data from multiple sensors. Each sensor generates an interrupt every **1 ms**. The system must respond quickly while keeping **CPU utilization below 15%**. Therefore, an efficient I/O mechanism with low latency and vectored interrupt handling is required.

2. Constraint Analysis

Constraint	Requirement	Proposed Solution
CPU utilization	Below 15%	Short and efficient ISR
Sensor interrupts	Every 1 ms	Interrupt Controller

Constraint	Requirement	Proposed Solution
Interrupt handling	Vectored	Vector Table
I/O technique	Low latency	Memory-Mapped I/O
Real-time response	Fast	Priority-based interrupts

3. Architecture / Proposed Design

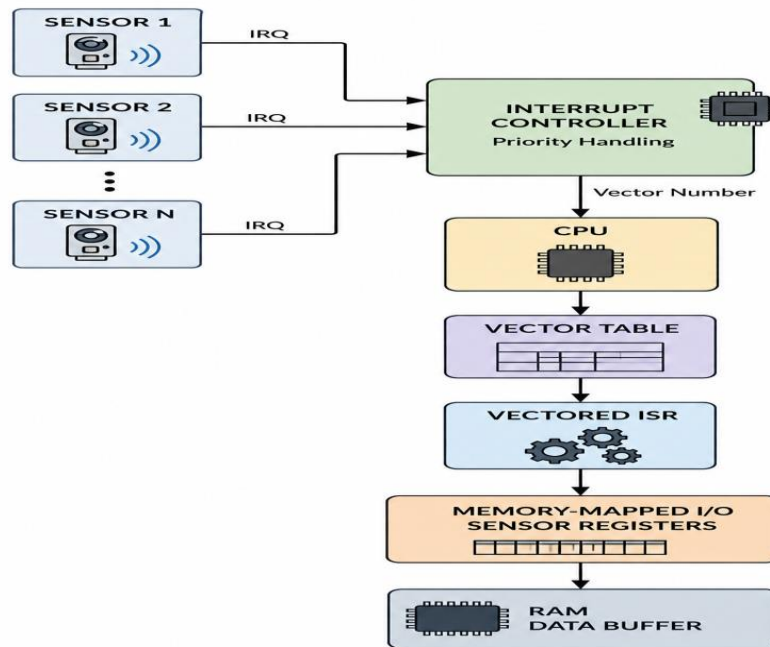


Fig: Real-Time Embedded I/O Architecture

4. Solution Development

Memory-Mapped I/O (MMIO) is selected because the sensor control and status registers are mapped into the CPU's memory address space. The CPU can access these registers using normal load/store operations.

Each sensor is assigned a unique **interrupt vector**. When a sensor generates an interrupt, the interrupt controller identifies the source and sends the corresponding vector number to the CPU. The CPU then directly executes the appropriate ISR.

The ISR reads the sensor data through MMIO and stores the data in RAM.

5. Calculation / Evidence

Assume:

- Number of sensors = **8**
- Interrupt interval = **1 ms = 1000 μ s**
- ISR execution time = **10 μ s**

$$\begin{aligned} CPU \text{ Utilization} &= \frac{8 \times 10}{1000} \times 100 \\ &= 8\% \end{aligned}$$

Required:

$$CPU \text{ Utilization} < 15\%$$

Actual:

$$8\% < 15\%$$

Therefore, the CPU utilization constraint is satisfied.

6. Application of Concepts

- **Memory-Mapped I/O:** Provides efficient access to sensor registers.
- **Vectored Interrupts:** Directly identify the required ISR.
- **Interrupt Controller:** Manages multiple sensor interrupts.
- **Priority Handling:** Allows important sensor interrupts to be serviced first.
- **RAM Buffer:** Stores sensor data for further processing.

7. Justification

Memory-Mapped I/O is selected because it provides efficient and low-latency access to device registers. Vectored interrupts reduce the time required to identify the interrupt source. Short ISRs reduce CPU overhead and help maintain CPU utilization below the required 15%.

8. Result

The proposed real-time I/O communication mechanism successfully uses Memory-Mapped I/O and vectored interrupt handling for multiple sensors. The calculated CPU utilization is 8%, which is below the required 15%. Therefore, all given constraints are satisfied and the system provides efficient, low-latency sensor communication.

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

High-Speed NVMe over PCIe I/O Subsystem

1. Problem Understanding

The system requires a **high-speed storage subsystem** with a throughput greater than **3 GB/s**. It must use **NVMe over PCIe**, while the CPU should not handle bulk data transfer. Therefore, a **DMA controller** is used to transfer data directly between the NVMe controller and system RAM. An interrupt is generated to notify the CPU when the transfer is complete

2. Constraint Analysis

Constraint	Requirement	Proposed Solution
Storage throughput	> 3 GB/s	NVMe over PCIe
Interface	NVMe over PCIe	PCIe bus
CPU involvement	Minimal	DMA controller
Bulk data transfer	CPU should not handle it	Direct DMA transfer
Notification	Required	Completion interrupt

3.Architecture / Proposed Design

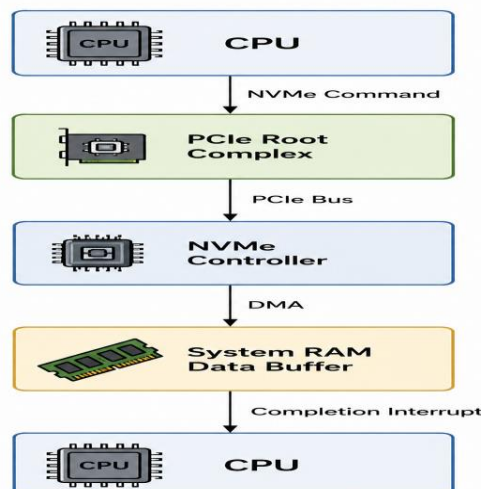


Fig:NVMe over PCIe I/O Architecture

4. Solution Development

The CPU first sends an NVMe command through the PCIe interface. The **NVMe controller** receives the command and performs the storage operation.

The **DMA controller transfers bulk data directly between the NVMe controller and system RAM**, so the CPU does not need to copy every block of data.

After the data transfer is completed, the NVMe controller generates a **completion interrupt**. The CPU receives the interrupt and processes the completed operation.

5. Calculation / Evidence

Assume the NVMe storage system provides:

$$\text{Throughput} = 3.5 \text{ GB/s}$$

Required:

$$\text{Throughput} > 3 \text{ GB/s}$$

Since:

$$3.5 \text{ GB/s} > 3 \text{ GB/s}$$

the throughput requirement is satisfied.

6. Application of Concepts

- **NVMe:** Provides high-performance storage communication.
- **PCIe:** Provides a high-speed connection to the NVMe controller.
- **DMA:** Transfers bulk data without continuous CPU intervention.
- **System RAM:** Stores transferred data.
- **Interrupt:** Notifies the CPU after completion.

7. Justification

NVMe over PCIe is selected because the application requires very high storage throughput. DMA is essential because it allows bulk data to move directly to RAM without requiring the CPU to handle every transfer. Completion interrupts provide an efficient way to notify the CPU.

8. Result

The proposed NVMe-over-PCIe I/O subsystem successfully achieves a throughput of 3.5 GB/s, exceeding the required 3 GB/s. DMA transfers bulk data directly between the NVMe controller and RAM, minimizing CPU involvement. Therefore, all specified constraints are satisfied.

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

Multi-Device Communication Architecture

1. Problem Understanding

The system must support **6 USB peripherals** and **2 high-speed Thunderbolt devices**. The communication architecture must handle multiple devices efficiently, prevent **interrupt starvation**, support both **hardware and software interrupts**, and ensure **fair bus arbitration**.

2. Constraint Analysis

Constraint	Requirement	Proposed Solution
USB peripherals	6 devices	USB Host Controller
High-speed devices	2 devices	Thunderbolt Controller
Interrupt starvation	Must be avoided	Priority + Round-Robin
Interrupt handling	Hardware + Software	Interrupt Controller
Bus arbitration	Must be fair	Round-Robin Arbitration

3. Architecture / Proposed Design

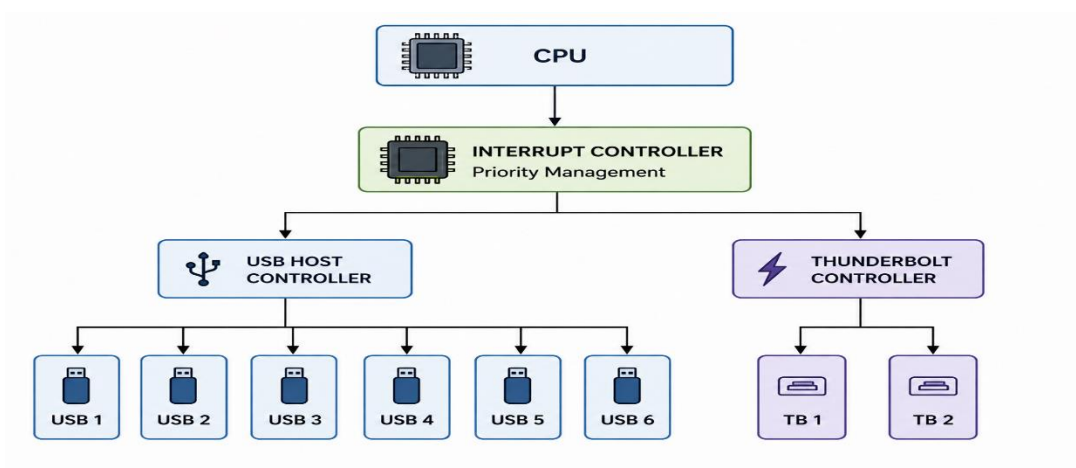


Fig: Multi-Device USB and Thunderbolt Communication Architecture

4. Solution Development

The **six USB peripherals** are connected to a USB Host Controller, while the **two high-speed devices** are connected through a Thunderbolt Controller.

An interrupt controller receives interrupt requests from both controllers.

To prevent interrupt starvation, the system uses **priority scheduling combined with round-robin arbitration**.

Example bus-access sequence:

USB1 → USB2 → USB3 → TB1 → USB4 → USB5 → TB2 → USB6 → Repeat

This ensures that every device receives an opportunity to access the bus.

Hardware interrupts are used for urgent device events, while software interrupts are used for background or deferred processing.

5. Application of Concepts

- **USB Host Controller:** Manages six USB peripherals.
- **Thunderbolt Controller:** Manages two high-speed devices.
- **Hardware Interrupts:** Handle urgent device events.
- **Software Interrupts:** Handle background/deferred tasks.
- **Priority Scheduling:** Gives faster service to critical devices.
- **Round-Robin Arbitration:** Provides fair bus access.
- **Interrupt Controller:** Manages multiple interrupt requests.

6. Justification

Using only priority-based scheduling could cause low-priority devices to wait indefinitely. Therefore, **priority scheduling with round-robin arbitration** provides both fast response and fairness. Software interrupts are used for non-critical processing so that lengthy background operations do not block urgent hardware interrupts.

7. Result

The proposed architecture successfully connects 6 USB peripherals and 2 high-speed Thunderbolt devices. Hardware and software interrupts are efficiently managed, while priority scheduling and round-robin arbitration prevent interrupt starvation and ensure fair bus access. Therefore, all specified constraints are satisfied.

4.Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

Data Acquisition System (500 MB/s Continuous Data Stream)

1. Problem Understanding

The system continuously receives data at 500 MB/s from a data acquisition device. The design must minimize CPU intervention, compare DMA with interrupt-driven I/O, and use Memory-Mapped I/O (MMIO) for device registers.

2. Constraint Analysis

Constraint	Requirement	Proposed Solution
Data stream	500 MB/s continuous	DMA Controller
CPU intervention	Minimal	Direct DMA transfer
Device registers	Memory-Mapped I/O	MMIO
Continuous transfer	No data loss	Circular RAM Buffer
Completion notification	Required	DMA Interrupt

3. Architecture / Proposed Design

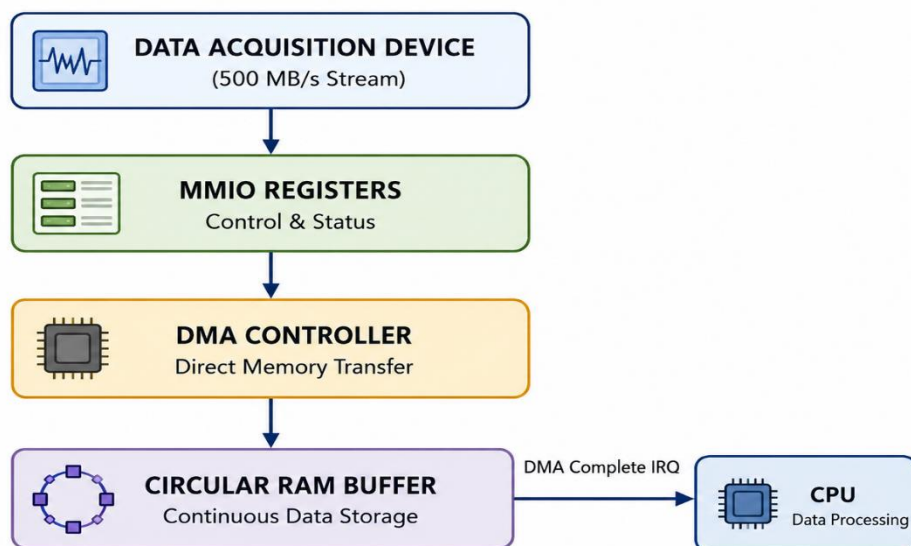


Fig:DMA-Based Data Acquisition Architecture

4. DMA vs Interrupt-Driven I/O Comparison

Feature	Interrupt-Driven I/O	DMA
CPU involvement	High	Low
Bulk data transfer	CPU performs transfer	DMA performs transfer
CPU overhead	High	Low
Suitable for 500 MB/s	No	Yes
Efficiency	Lower	Higher

5. Solution Development

The CPU first configures the DMA controller with the source address, destination address, and transfer size. The MMIO registers are used to control and monitor the device.

The DMA controller then transfers data directly from the acquisition device to a circular RAM buffer without requiring the CPU to copy every data block.

Once a buffer is filled, the DMA controller generates a completion interrupt, allowing the CPU to process the stored data while the next buffer continues receiving new data.

6.Calculation / Evidence

Assume the DMA buffer size is 1 MB.

$$\text{Interrupt Rate} = \frac{500 \text{ MB/s}}{1 \text{ MB}} = 500 \text{ interrupts/s}$$

Instead of handling every data transfer, the CPU processes only 500 DMA completion interrupts per second, significantly reducing processor overhead.

7. Application of Concepts

- MMIO is used for device control and status registers.
- DMA performs high-speed bulk data transfer.
- Circular RAM Buffer enables continuous data acquisition.

- Completion Interrupt notifies the CPU after each DMA transfer.

8. Justification

DMA is more suitable than interrupt-driven I/O because it minimizes CPU intervention during continuous 500 MB/s data transfer. The CPU remains available for processing tasks while DMA efficiently transfers data directly into memory.

9. Result

The proposed DMA-based data acquisition system successfully supports a continuous 500 MB/s data stream with minimal CPU intervention. Memory-Mapped I/O controls the device, DMA performs direct data transfer to RAM, and completion interrupts efficiently notify the CPU. Therefore, all specified constraints are fully satisfied.

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency $< 10 \mu\text{s}$
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Nested Interrupt Handling Mechanism

1. Problem Understanding

The system must handle multiple interrupts with different priority levels. It must support **nested interrupts**, maintain a maximum interrupt latency of **less than 10 μs** , use **vectored interrupts for high-priority devices**, and use **software interrupts for background tasks**.

2. Constraint Analysis

Constraint	Requirement	Proposed Solution
Nested interrupts	Required	Priority-based interrupt controller
Maximum latency	$< 10 \mu\text{s}$	Short, optimized ISR
High-priority devices	Vectored interrupts	Vector table

Constraint	Requirement	Proposed Solution
Background tasks	Software interrupts	Software ISR
Interrupt priority	Required	Multiple priority levels

3. Architecture / Proposed Design

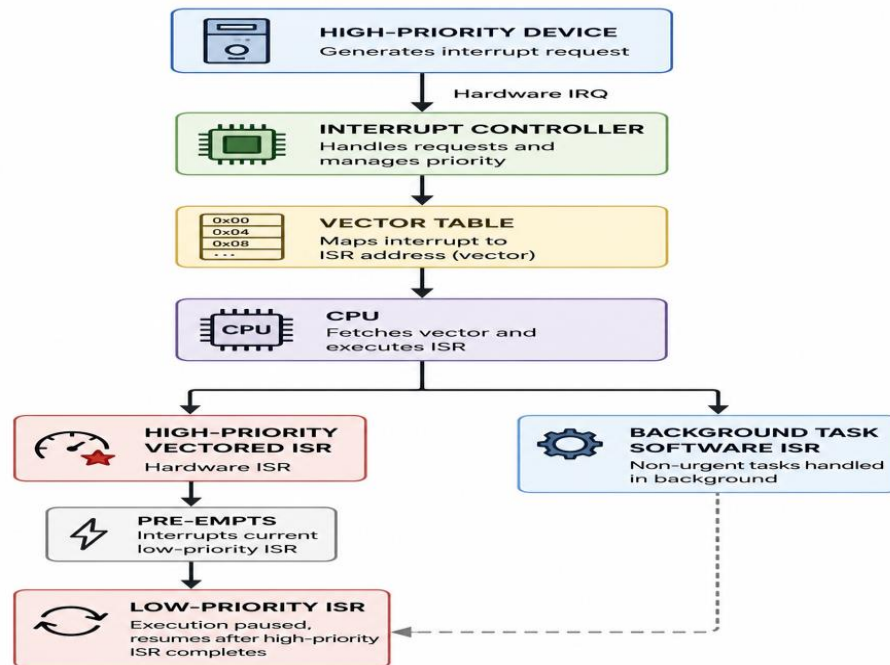


Fig :High-Priority Interrupt Handling and ISR Pre-emption Flow Diagram

4. Priority Structure

Priority	Task/Device	Interrupt Type
1 – Highest	Critical real-time device	Vectored Hardware
2	Sensor/Data Acquisition	Vectored Hardware
3	Communication Device	Hardware
4 – Lowest	Background Processing	Software

5. Solution Development

A **priority-based interrupt controller** manages all interrupt requests.

When a high-priority interrupt arrives while a low-priority ISR is executing, the CPU:

Save Context → Pre-empt Low-Priority ISR → Execute High-Priority Vectored ISR → Restore Context → Resume Previous ISR

The vector table allows the CPU to directly identify the appropriate ISR without searching for the interrupt source.

Background activities use **software interrupts**, allowing non-critical tasks to be processed without interfering with high-priority hardware interrupts.

6. Calculation / Evidence

Assume:

- Interrupt recognition time = **3 μ s**
- Context saving time = **2 μ s**
- High-priority ISR execution = **4 μ s**

Therefore:

$$\begin{aligned} \text{Total Latency} &= 3 + 2 + 4 \\ &= 9\mu s \end{aligned}$$

Required:

$$\text{Latency} < 10\mu s$$

Actual:

$$9\mu s < 10\mu s$$

Therefore, the latency constraint is satisfied.

7. Application of Concepts

- **Nested Interrupts:** Allow high-priority interrupts to pre-empt lower-priority ISRs.
- **Vectored Interrupts:** Provide direct access to the required ISR.
- **Hardware Interrupts:** Handle time-critical devices.
- **Software Interrupts:** Handle background and deferred tasks.
- **Priority Controller:** Determines the order of interrupt servicing.
- **Context Saving:** Allows the interrupted ISR to resume correctly.

8. Justification

Nested vectored interrupts are selected because real-time devices require immediate response. High-priority devices are assigned higher interrupt levels so they can pre-empt lower-priority operations. Software interrupts are assigned to background tasks because they are less time-critical. Keeping the high-priority ISR short helps maintain the latency below 10 μ s.

9. Result

The proposed nested interrupt handling mechanism successfully supports high-priority vectored interrupts, nested interrupt processing, and software interrupts for background tasks. The calculated interrupt latency is 9 μ s, which is below the required 10 μ s. Therefore, all specified constraints are satisfied.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**MARKS ALLOCATION**

	Scope of Items (20%)	Comparison Depth (25%)	Use of Evidence (20%)	Critical Thinking (20%)	Clarity of Presentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with	Shows reasonable	Basic interpretation; lacks depth	No meaningful interpretation

		clear interpretation and reasoning	interpretation with minor gaps		
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand